Reverse Engineering a Packed Trojan: A Ghidra and x64dbg Case Study

Executive Summary

The analyzed executable is a **highly sophisticated Windows malware** employing multiple layers of **obfuscation, self-modifying code, and runtime unpacking** to evade detection and hinder analysis. It manually reconstructs its own Portable Executable (PE) structures in memory, dynamically resolves APIs, and uses non-standard sections—clear indicators of intentional anti-analysis design.

The malware enables **high-privilege access**, enforces **single-instance execution**, and adapts its behavior based on execution context and command-line arguments. It establishes **persistence through repeated execution logic**, background threads, and controlled relaunch mechanisms. Network-related functionality and dynamic URL generation indicate **command-and-control capabilities**, while long sleep loops and conditional execution suggest **sandbox and analysis evasion**.

Overall, this sample is **not commodity malware**; it aligns with tooling used for **long-term access, stealthy execution, and controlled remote operations**, posing a **high risk to affected systems and environments**.

Analysis Environment

- Operating system: Windows 10 Flare VM, Macos
- Tools used:
  - Ghidra
  - x64dbg
  - PE tools (Pestudio, Detect It Easy, PEiD, etc.)

Sample Information

Basic File Properties

Category	Attribute	Value
Hashes	MD5	`69f27b07404cf9c51dd2d2e40fca4d65`
	SHA-1	`56d7f4aadedbabf9b889dd6fb39f710812fee09e`
	SHA-256	`5617238B8D3B232F0743258B89720BB04D941278253E841EE9CBF863D0985C32`
	Imphash	`4de321e101993eb64afa7391ff67dfd0`
	Authentihash	`497c847e3d77c6d05e9ead560708f151fa5f53c6a2e46baef5730746d1eb99f4`
	Rich PE Header Hash	`78e47bdfe058f62eed025b9cc4f5e4fe`
	SSDEEP	`6144:wJvWo7JhhqFTkptLgBX74A19+cMBmOjHEtDYWxcGrKOwLByOBveZ+gw/TpTQ03la:SNd4TOghZ9d0mTiWALJ`
	TLSH	`T15005C25F2F81C308F2A92DF6D5D5D8F2027F71DF5E28686691D8106CC6AA29CFA36350`
	Vhash	`0850b6551d151517171710292z5nz1fz`
File Properties	File Name	`69f27b07404cf9c51dd2d2e40fca4d65.bin`
	File Size	858,112 bytes (838.00 KB)
	File Type	Win32 Executable
	Architecture	PE32 (32-bit, GUI)
	Operating System	Microsoft Windows

Category	Attribute	Value
	Entropy	5.729
	Version	4.20.0
	Description	Command line RAR
Headers	Magic Bytes (Hex)	`4D 5A 90 00 03 00 00 00 04 00 00 00 FF FF 00 00 B8 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00`
	Magic Bytes (Text)	`MZ.....@.....`
Entry Point	Address	`0x00001200`
	Section	`.text`
	First 32 Bytes (Hex)	`55 8B EC 83 EC 14 57 C7 45 FC 44 A0 4C 00 A1 18 60 4C 00 89 45 F4 68 2D 14 00 00 6A 00 FF 15 24`
Toolchain	Compiler	Microsoft Visual C/C++
	Linker	Microsoft Linker 9.0
	Toolset	Visual Studio 2008
	Build Timestamp	Fri Feb 13 00:37:48 2015 (UTC)
Signature & Metadata	PE Signature	Microsoft Linker 9.0
	Manifest	WinRAR
	Debug Symbols	Not Present
	Export Table	Not Present
	Digital Certificate	Not Present
.NET	Module Name	Not Applicable

PE HEADER

Field	Value
Target Machine	Intel 386 or later processors (x86)
Compilation Timestamp	2015-02-13 00:37:48 UTC
Entry Point (RVA)	0x1200 (4608)
Number of Sections	11

PE SECTION

Name	Virtual Address	Virtual Size	Raw Size	Entropy	MD5	Ch
.text	0x1000	806,833	806,912	5.79	0ec63edf45aa2a22b5e4e81e3312b088	22
.rdata	0xC6000	12,340	12,800	1.44	4b94c951b51301456e7e445876c10122	2,4
.data	0xCA000	564	512	1.49	74823794b08fc243d186c87914870619	90
.t22112	0xCB000	555	1,024	0.99	4ea215b1fcfd8271254460ed7c8f851b	13(
.t2211	0xCC000	555	1,024	0.99	4ea215b1fcfd8271254460ed7c8f851b	13(
.t221	0xCD000	555	1,024	0.99	4ea215b1fcfd8271254460ed7c8f851b	13(
.t22	0xCE000	555	1,024	0.99	4ea215b1fcfd8271254460ed7c8f851b	13(
.t21	0xCF000	555	1,024	0.99	4ea215b1fcfd8271254460ed7c8f851b	13(
.t2	0xD0000	555	1,024	0.99	4ea215b1fcfd8271254460ed7c8f851b	13(
.rdat	0xD1000	1,000	1,024	0.16	01cb5594d3b2c332c81196b782ade080	24
.rsrc	0xD2000	29,232	29,696	3.49	516c129e8650fabf0816b135bc84f78c	2,1

Packing & Obfuscation Assessment

The analyzed sample exhibits strong indicators of packing or custom obfuscation.

Supporting Evidence

- The executable contains **multiple non-standard sections** (`.t22112`, `.t2211`, `.t221`, `.t22`, `.t21`, `.t2`) that are **not produced by standard Windows compilers or linkers**.
- These sections have:
  - **Identical sizes**
  - **Identical MD5 hashes**
  - **Very low entropy**
- The presence of repeated sections with identical content is **highly indicative of packer-generated padding or anti-analysis artifacts**.
- The `.text` section is **abnormally large (~807 KB)** with **moderate entropy (5.79)**, suggesting:
  - The payload may already be **decompressed or decrypted early**
  - The sample likely uses a **lightweight or custom unpacking routine** rather than full encryption
- The **entry point** resides in the `.text` section and appears consistent with **loader or initialization logic**, commonly observed in packed trojans.
- File metadata and resources masquerade as **legitimate WinRAR components**, while the internal structure **does not match known clean WinRAR binaries**, further supporting trojanization.

Description

Despite presenting itself as a legitimate WinRAR command-line executable, the sample's context, size, and subsequent behavior indicate potential trojanization or use as a packed carrier, justifying further unpacking and dynamic analysis using x64dbg and Ghidra.

Imported APIs

Function Name	Import Type	Thunk Type	Hint	RVA	VA	DLL
LoadLibraryA	Implicit	–	–	0x000C8FCC	0x000C8FCC	KERNEL32.dll
GetProcAddress	Implicit	–	–	0x000C8FBA	0x000C8FBA	KERNEL32.dll
GetModuleHandleW	Implicit	–	–	0x000C8FA6	0x000C8FA6	KERNEL32.dll
CreateFileW	Implicit	–	–	0x000C8F98	0x000C8F98	KERNEL32.dll
GetDriveTypeW	Implicit	–	–	0x000C8F88	0x000C8F88	KERNEL32.dll
LoadCursorA	Implicit	–	–	0x000C8FEA	0x000C8FEA	USER32.dll
RegOpenKeyA	Implicit	–	–	0x000C9004	0x000C9004	ADVAPI32.dll
RegQueryValueExA	Implicit	–	–	0x000C9012	0x000C9012	ADVAPI32.dll

---

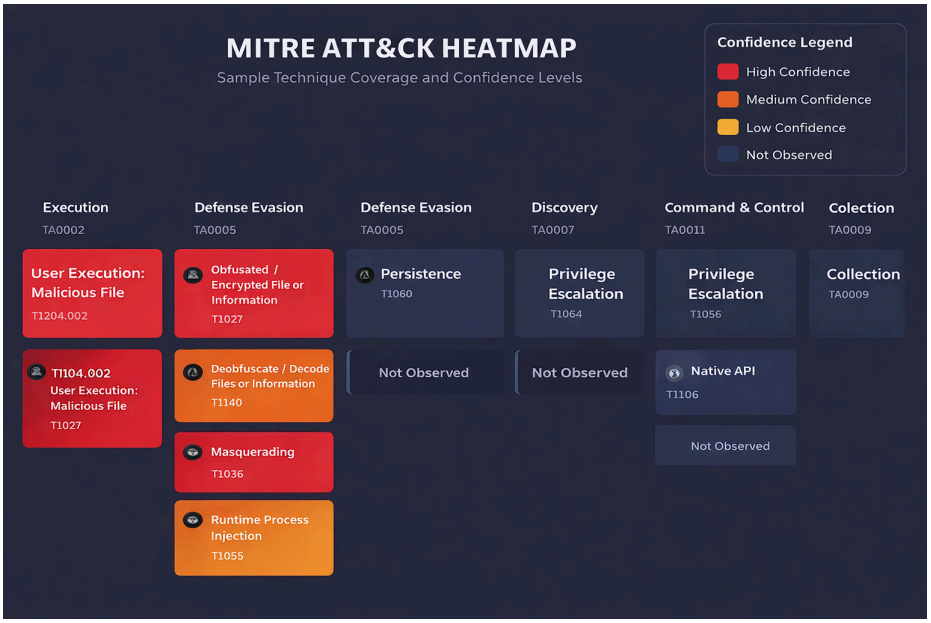
Context

- **Dynamic API resolution** (`LoadLibraryA`, `GetProcAddress`) suggests runtime linking and potential evasion.
- **Registry access** APIs indicate system or configuration inspection.
- **Drive enumeration** (`GetDriveTypeW`) may be used for environment awareness or anti-analysis checks.
- **User32 interaction** is minimal, possibly for disguise or resource loading.

MITRE ATT&CK Mapping

Tactic	Technique ID	Technique Name	Evidence
Defense Evasion	**T1027**	Obfuscated / Encrypted File or Information	Use of non-standard and repeated PE sections (`.t221*`) with identical hashes and

Tactic	Technique ID	Technique Name	Evidence
			abnormal layout indicates deliberate obfuscation to hinder static analysis.
Defense Evasion	**T1140**	Deobfuscate / Decode Files or Information	Loader-style entry point and large `.text` section suggest runtime decoding or unpacking of the payload.
Defense Evasion	**T1055**	Process Injection *(Potential)*	Packing behavior commonly precedes injection techniques; dynamic analysis is required to confirm runtime code injection.
Defense Evasion	**T1036**	Masquerading	File metadata and resources impersonate legitimate WinRAR components while internal structure deviates from clean binaries.
Execution	**T1204.002**	User Execution: Malicious File	Distributed as a Windows GUI executable requiring user interaction to execute.



Ghidra Analysis

Entry Function Analysis

```
/* WARNING: Globals starting with '_' overlap smaller symbols at the same address */

undefined4 __cdecl entry(undefined4 param_1)

{
```

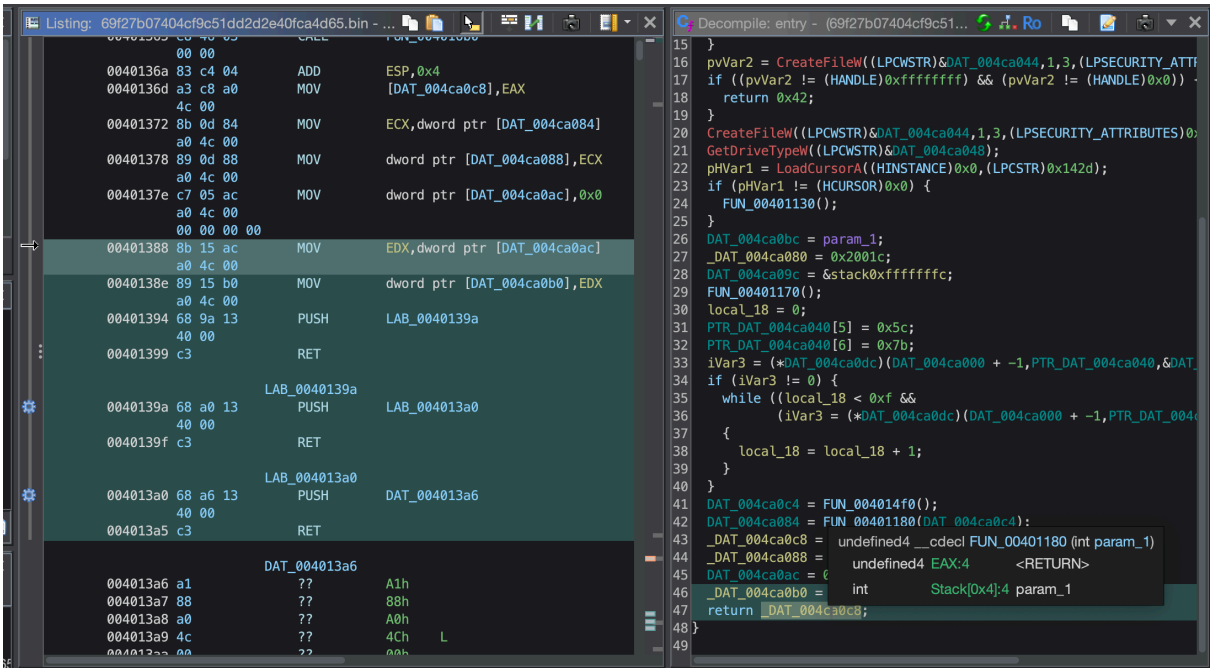
```
  HCURSOR pHVar1;
  HANDLE pvVar2;
  int iVar3;
  uint local_18;

  pHVar1 = LoadCursorA((HINSTANCE)0x0,(LPCSTR)0x142d);
  if (pHVar1 != (HCURSOR)0x0) {
    FUN_00401130();
  }
  pvVar2 = CreateFileW((LPCWSTR)&DAT_004ca044,1,3,(LPSECURITY_ATTRIBUTES)0x0,3,0x80,(HANDLE)0x0);
  if ((pvVar2 != (HANDLE)0xffffffff) && (pvVar2 != (HANDLE)0x0)) {
    return 0x42;
  }
  CreateFileW((LPCWSTR)&DAT_004ca044,1,3,(LPSECURITY_ATTRIBUTES)0x0,3,0x80,(HANDLE)0x0);
  GetDriveTypeW((LPCWSTR)&DAT_004ca048);
  pHVar1 = LoadCursorA((HINSTANCE)0x0,(LPCSTR)0x142d);
  if (pHVar1 != (HCURSOR)0x0) {
    FUN_00401130();
  }
  DAT_004ca0bc = param_1;
  _DAT_004ca080 = 0x2001c;
  DAT_004ca09c = &stack0xfffffffc;
  FUN_00401170();
  local_18 = 0;
  PTR_DAT_004ca040[5] = 0x5c;
  PTR_DAT_004ca040[6] = 0x7b;
  iVar3 = (*DAT_004ca0dc)(DAT_004ca000 + -1,PTR_DAT_004ca040,&DAT_004ca22c);
  if (iVar3 != 0) {
    while ((local_18 < 0xf &&
           (iVar3 = (*DAT_004ca0dc)(DAT_004ca000 + -1,PTR_DAT_004ca040,&DAT_004ca22c), iVar3 != 0)))
    {
      local_18 = local_18 + 1;
    }
  }
  DAT_004ca0c4 = FUN_004014f0();
  DAT_004ca084 = FUN_00401180(DAT_004ca0c4);
  _DAT_004ca0c8 = FUN_004016b0(DAT_004ca084);
  _DAT_004ca088 = DAT_004ca084;
  DAT_004ca0ac = 0;
  _DAT_004ca0b0 = 0;
  return _DAT_004ca0c8;
}
```

The entry function acts as a loader rather than an application entry, performing environment checks, execution gating, and staged payload preparation. No legitimate WinRAR functionality is executed at this stage, confirming the executable is trojanized.

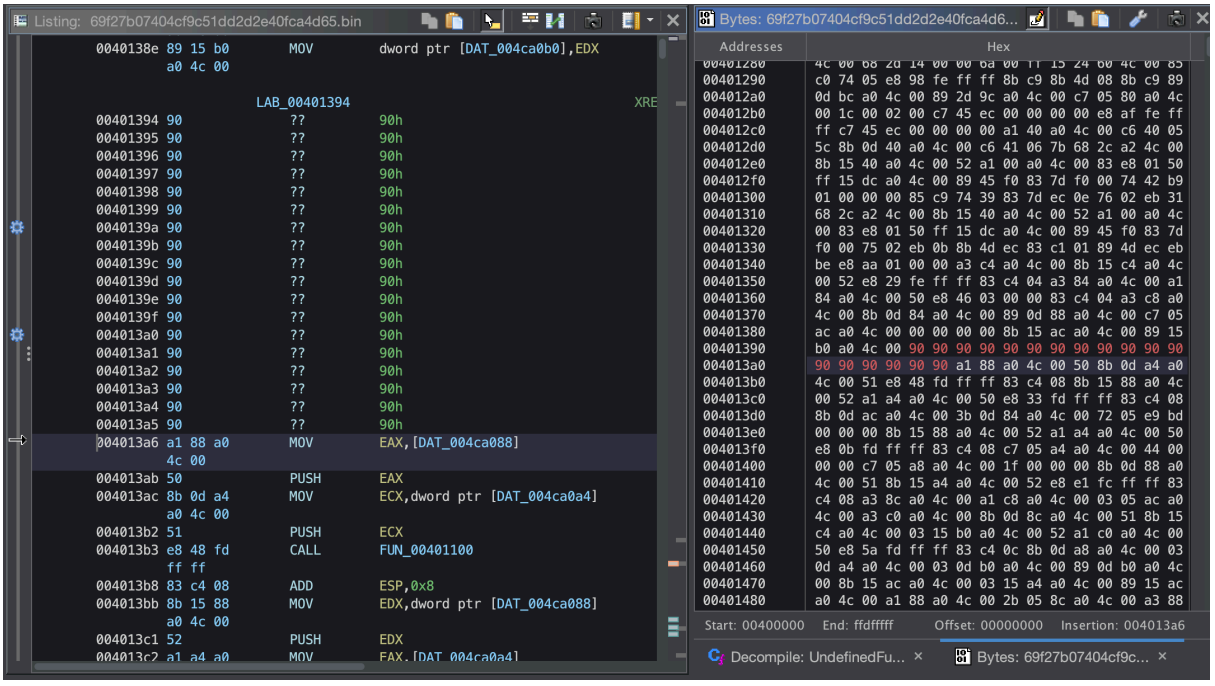
Control Flow Obfuscation via RET Trampolines

Ghidra analysis identified multiple instances of control flow redirection implemented using PUSH/RET instruction sequences rather than direct JMP or CALL instructions. These constructs form chained RET trampolines that ultimately redirect execution to a dynamically resolved address stored in a data region.

This technique intentionally disrupts static control flow analysis and is commonly observed in obfuscated loaders and unpacking stubs.



Overwriting the the bytes with nop



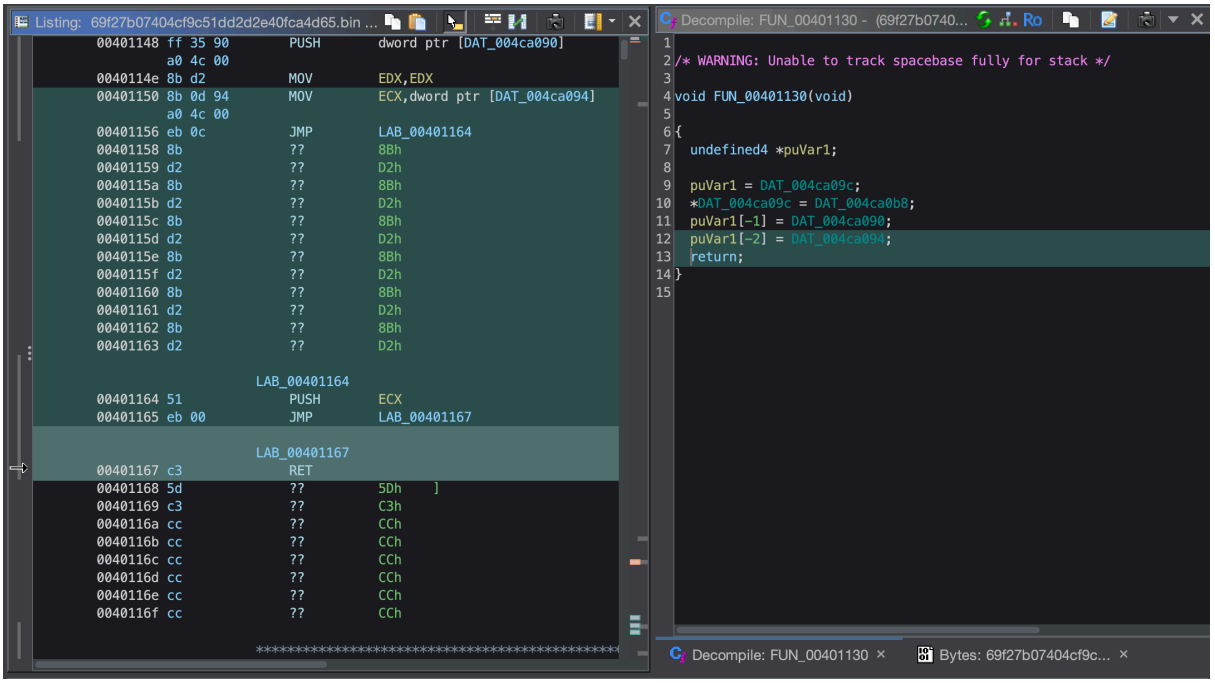
Obfuscated Indirect Control Flow

Static analysis of this code region reveals deliberate control-flow obfuscation combined with anti-disassembly techniques.

The function first loads a value from a global variable into the `ECX` register and immediately transfers execution to a later basic block, skipping over a sequence of bytes that are never executed. The skipped bytes consist of repeated `mov edx, edx` instructions, which act as NOP-equivalent operations and serve solely as dead code intended to confuse linear disassembly and control-flow reconstruction.

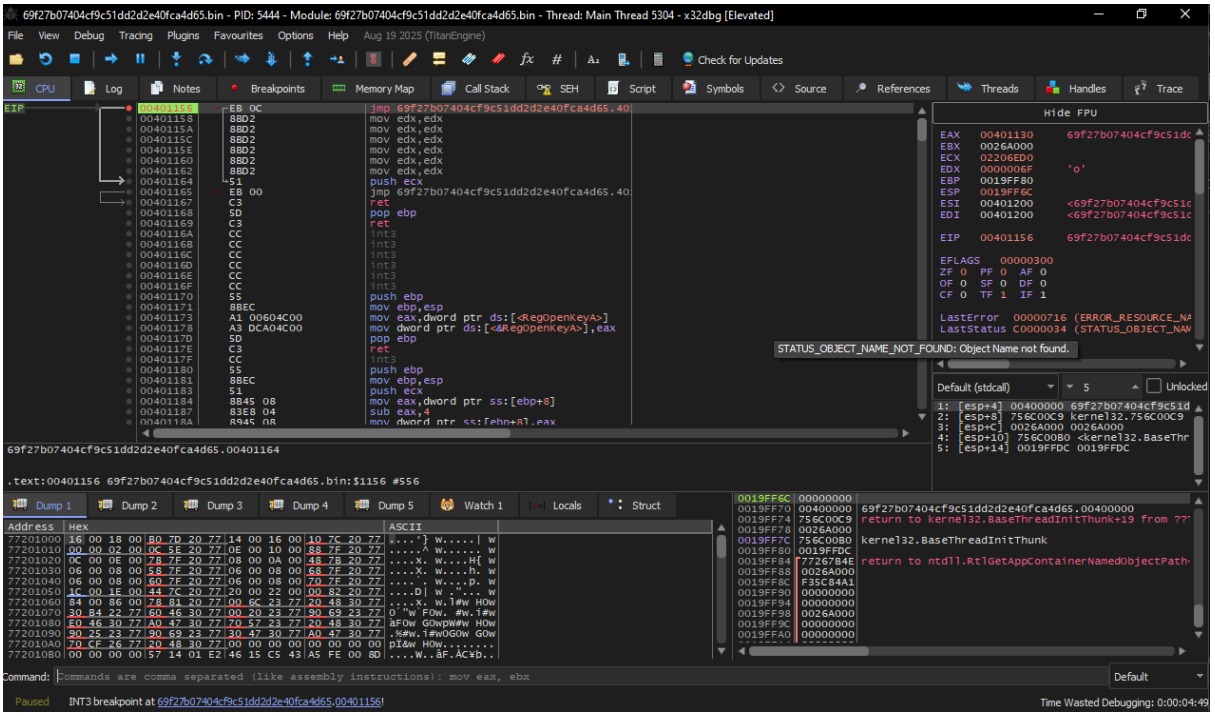
Execution then performs a `push ecx` followed by a `ret` instruction. Rather than returning to a caller, this sequence causes execution to continue at the address stored in `ECX`, effectively implementing an indirect jump to a runtime-resolved address.

This technique avoids the use of conventional `jmp` or `call` instructions and deliberately obscures execution flow. Such RET-based indirect jumps and dead code insertion are commonly observed in obfuscated malware loaders and unpacking stubs and are inconsistent with legitimate compiler-generated code.

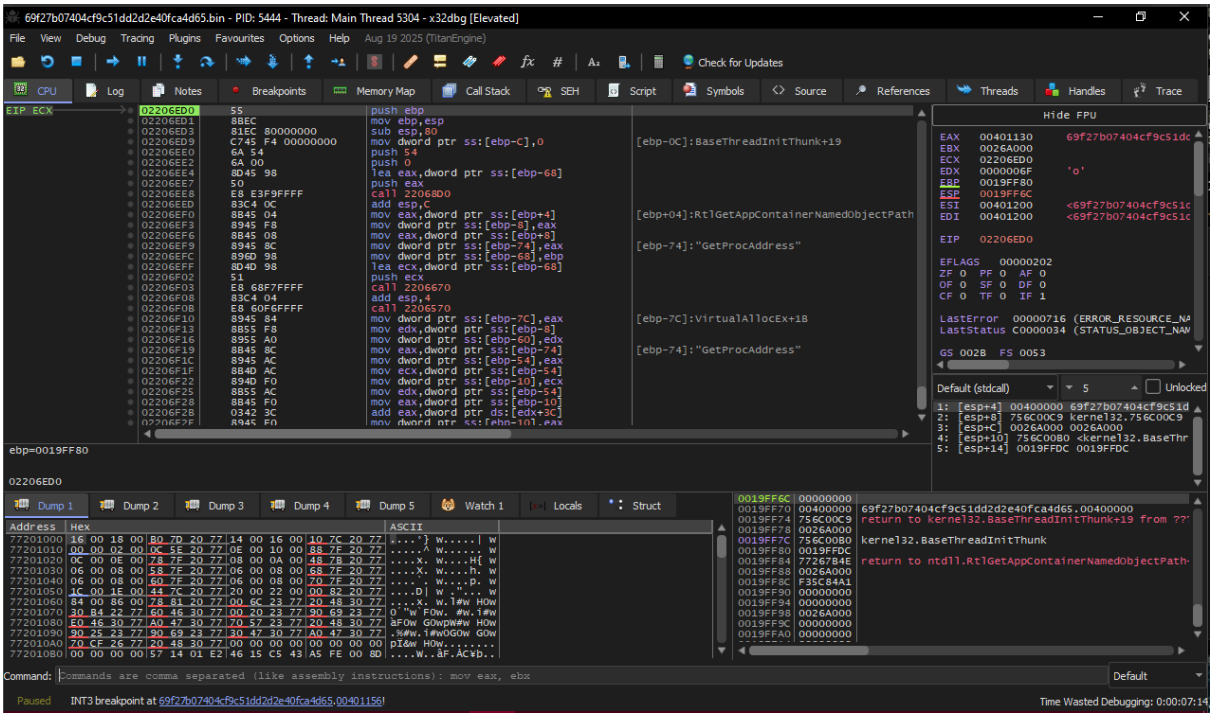


X32 DBG Analysis

Set a breakpoint at 00401156

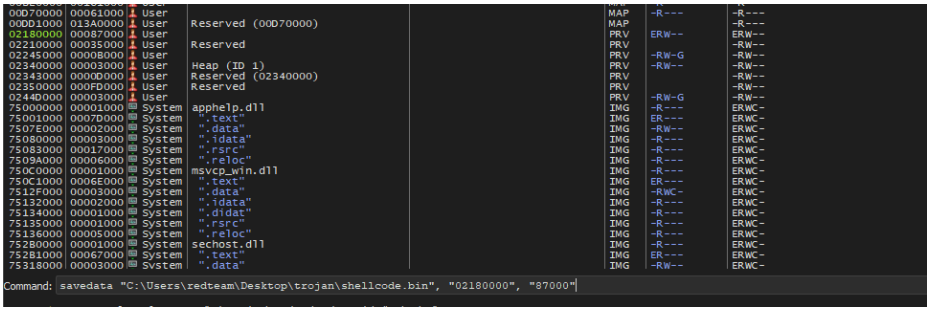


Stepping over

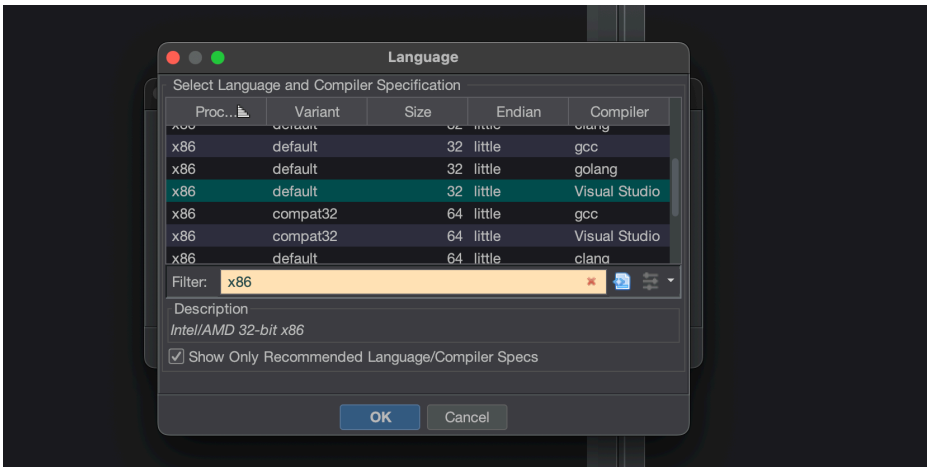


Base allocation and size.

Extracting shellcode



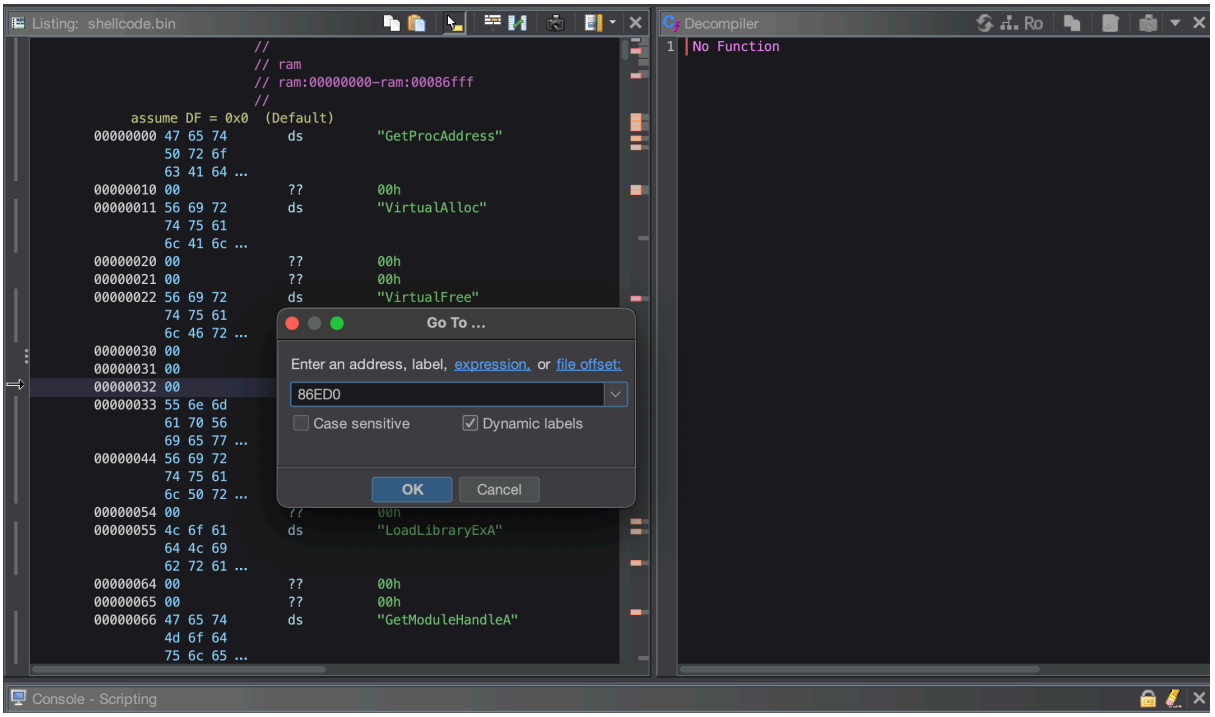
Loading extracted shellcode in ghidra

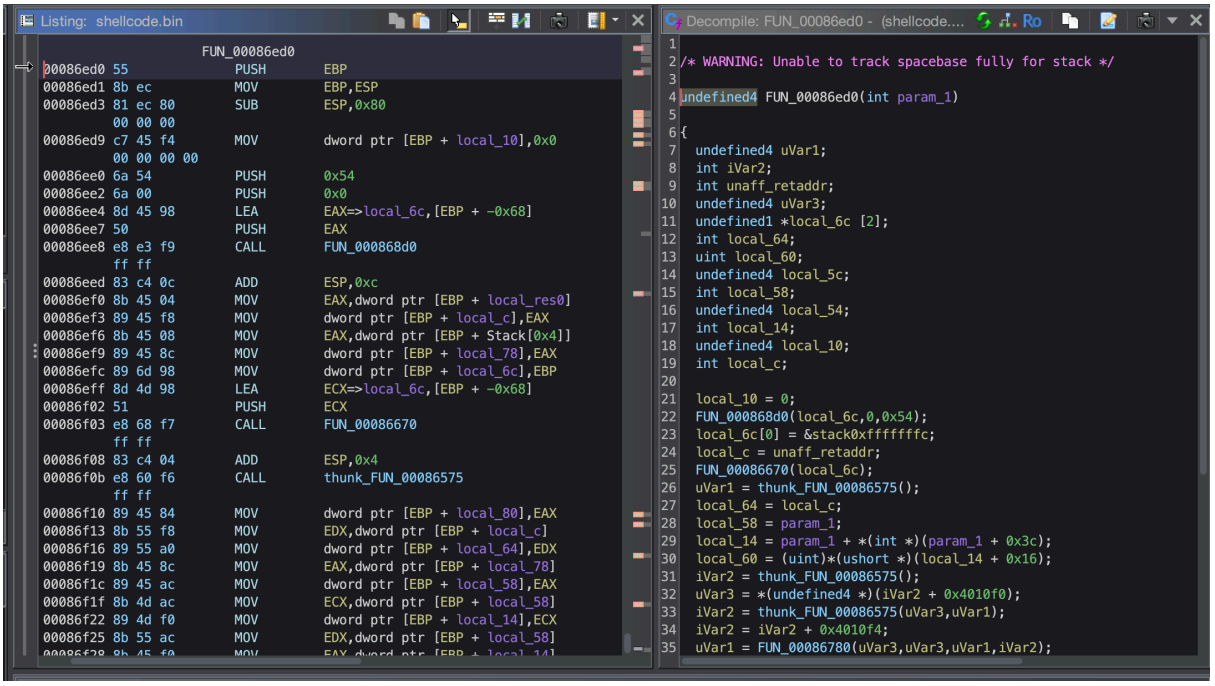


Offset

We need to find the offset subtract base allocation from current location.

0x02206ED0 - 0x2180000 = 0x86ED0

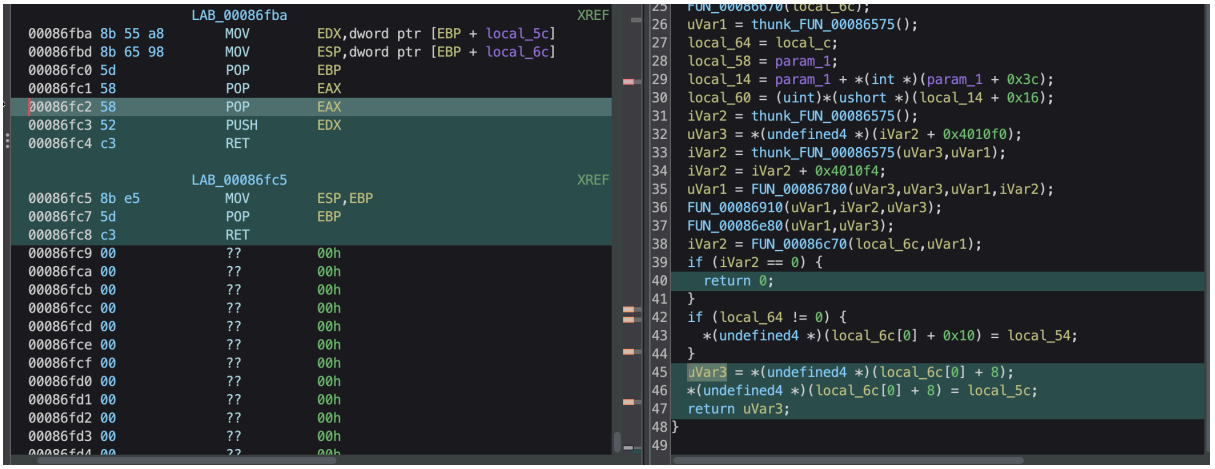





```
/* WARNING: Unable to track spacebase fully for stack */

undefined4 FUN_00086ed0(int param_1)

{
  undefined4 uVar1;
  int iVar2;
  int unaff_retaddr;
  undefined4 uVar3;
```

```
  undefined1 *local_6c [2];
  int local_64;
  uint local_60;
  undefined4 local_5c;
  int local_58;
  undefined4 local_54;
  int local_14;
  undefined4 local_10;
  int local_c;

  local_10 = 0;
  FUN_000868d0(local_6c,0,0x54);
  local_6c[0] = &stack0xfffffffc;
  local_c = unaff_retaddr;
  FUN_00086670(local_6c);
  uVar1 = thunk_FUN_00086575();
  local_64 = local_c;
  local_58 = param_1;
  local_14 = param_1 + *(int *)(param_1 + 0x3c);
  local_60 = (uint)*(ushort *)(local_14 + 0x16);
  iVar2 = thunk_FUN_00086575();
  uVar3 = *(undefined4 *)(iVar2 + 0x4010f0);
  iVar2 = thunk_FUN_00086575(uVar3,uVar1);
  iVar2 = iVar2 + 0x4010f4;
  uVar1 = FUN_00086780(uVar3,uVar3,uVar1,iVar2);
  FUN_00086910(uVar1,iVar2,uVar3);
  FUN_00086e80(uVar1,uVar3);
  iVar2 = FUN_00086c70(local_6c,uVar1);
  if (iVar2 == 0) {
    return 0;
  }
  if (local_64 != 0) {
    *(undefined4 *)(local_6c[0] + 0x10) = local_54;
  }
  uVar3 = *(undefined4 *)(local_6c[0] + 8);
  *(undefined4 *)(local_6c[0] + 8) = local_5c;
  return uVar3;
}
```

Reuse of Stack-Based Control Flow Obfuscation



This code fragment follows the same stack-based control flow redirection pattern observed earlier in the loader logic. Instead of using direct branch instructions, execution flow is altered through explicit manipulation of the return address on the stack.

The sequence `pop eax; push edx; ret` replaces the original return address with a value stored in the `EDX` register, causing execution to continue at a runtime-determined location. This mirrors previously observed `push <reg>; ret` constructs used to implement indirect jumps.

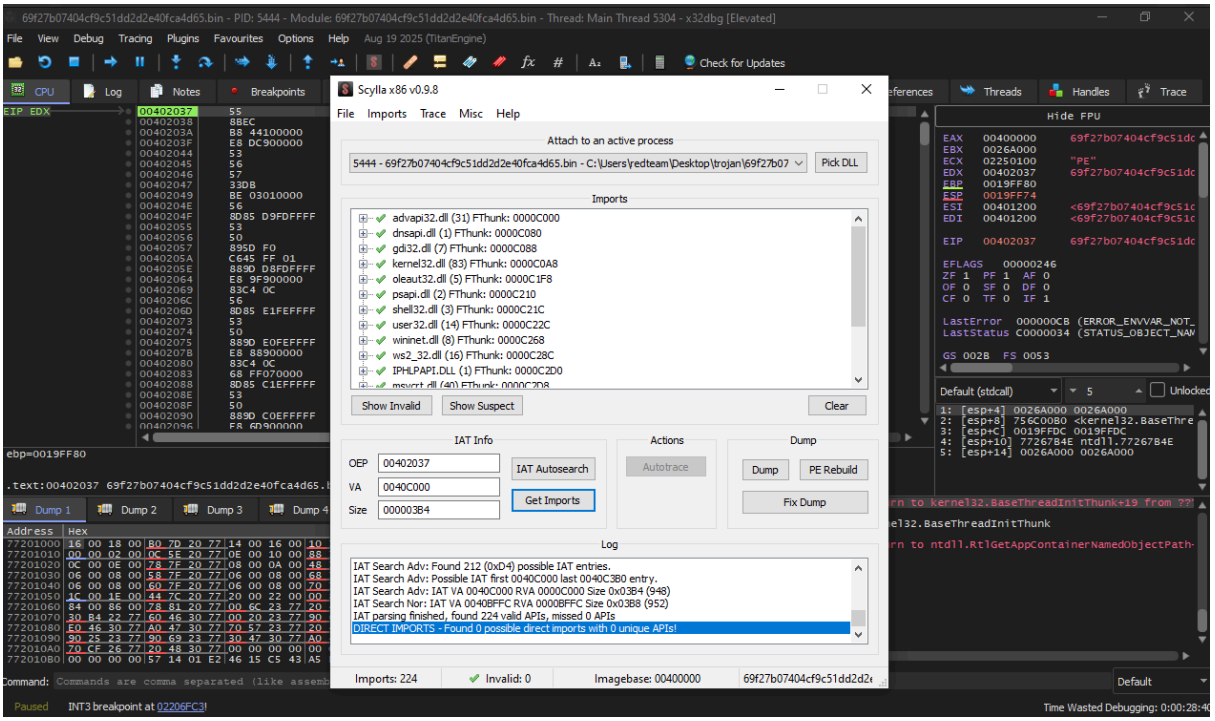
A secondary execution path performs a standard function epilogue (`mov esp, ebp; pop ebp; ret`), ensuring correct stack cleanup while maintaining obfuscated control flow.

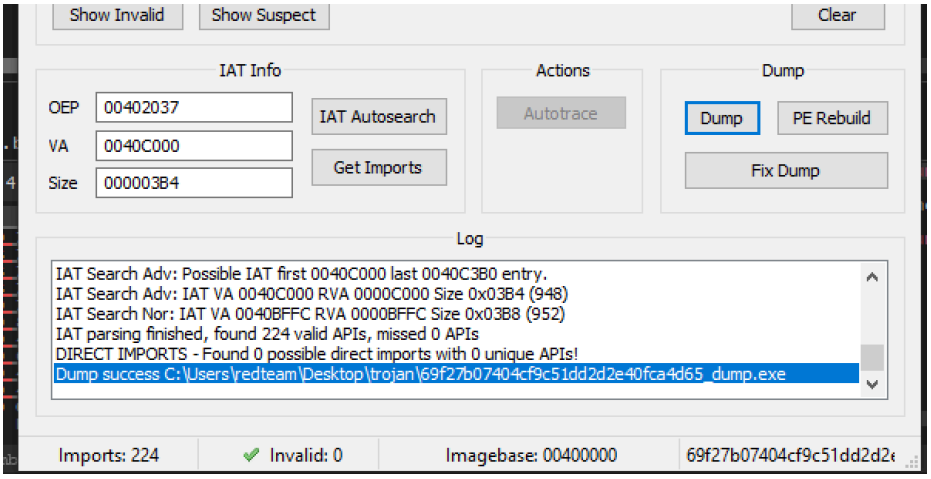
The repeated use of this technique across multiple stages indicates a deliberate and consistent **control-flow obfuscation strategy**, characteristic of custom loader or shellcode implementations and inconsistent with compiler-generated code.

```
00086fc1 58          POP     EAX
00086fc2 58          POP     EAX
00086fc3 52          PUSH    EDX
00086fc4 c3          RET
                 LAB_00086fc5          XREF[1]:    00086fa9(j)
00086fc5 8b e5       MOV     ESP,EBP
```

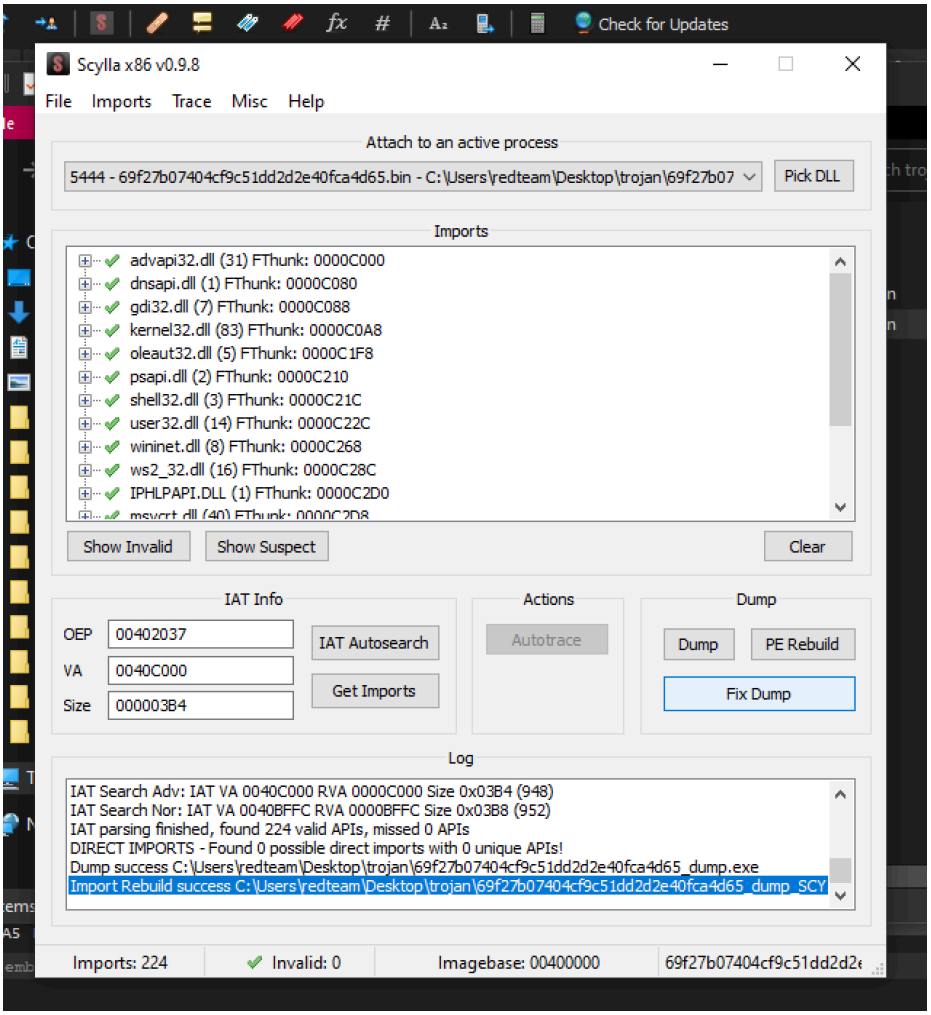
Dumping using Scylla

IAT auto search then get imports then dump it.





Fix dump



Analyzing Dump exe

Basic File Properties (Unpacked Payload)

Property	Value
File name	69f27b07404cf9c51dd2d2e40fca4d65_dump_SCY.exe
MD5	7a26ca74e95685f2f2922fcec617ef3e
SHA-1	f52bb228ebc49f7fb0d9de22cfe0e07ef29b80af
SHA-256	b15ff40974e537e77958d201b46d121a152a1ebb09d6d7b8057c1e2532a98570
Vhash	0850ce5d1d151517171711f5z70053281z71z205023z33ze1z80018z
Authentihash	ae9b916ea910183f81030f1bf7c645b3feccd2d507b2dca7414cb2576bd52b29
Imphash	66f92f54216d5fb2554b5d0a1bfe191f
Rich PE Header Hash	78e47bdfe058f62eed025b9cc4f5e4fe
SSDEEP	12288:cF/hs963OcOtC6n+loi8qojl6+rDMCe5cy+MCA1:+q63ut3+loikl6+fMC4r+MCA1
TLSH	T1D5053A4B3FF4C208F1A669F5D5F1A8E6077B75AF6D34642A81CC501CC7B2A88EA71391
File Type	Win32 EXE
Magic	PE32 executable (GUI) Intel 80386, for MS Windows
TrID	WinRAR Self-Extracting archive (4.x-5.x) (88.1%)Microsoft Visual C++ compiled executable (generic) (5.4%)Win64 Executable (generic) (3.5%)Win32 Executable (generic) (1.4%)Generic Win/DOS Executable (0.6%)
Detect It Easy	PE32Compiler: Microsoft Visual C/C++ (15.00.21022)Linker: Microsoft Linker (9.00.21022)Tool: Visual Studio 2008
Magika	PEBIN
File Size	843.50 KB (863,744 bytes)

PE HEADER

Field	Value
Target Machine	Intel 386 or later processors and compatible processors
Compilation Timestamp	2015-02-13 00:37:48 UTC
Entry Point (RVA)	8247
Contained Sections	12

PE SECTIONS

Name	Virtual Address	Virtual Size	Raw Size	Entropy	MD5	Chi²
.text	4096	806,912	806,912	5.55	5ed7c9837ba663f43d109f78f38de5ea	27,862
.rdata	811,008	16,384	12,800	1.45	a0b793ae54ee34ff35b8c77828f69d87	2,418,3
.data	827,392	4,096	1,024	1.65	10b1b398e5372456efc78a20c81210d0	178,92
.t22112	831,488	4,096	1,024	0.99	4ea215b1fcfd8271254460ed7c8f851b	130,97
.t2211	835,584	4,096	1,024	0.99	4ea215b1fcfd8271254460ed7c8f851b	130,97
.t221	839,680	4,096	1,024	0.99	4ea215b1fcfd8271254460ed7c8f851b	130,97
.t22	843,776	4,096	1,024	0.99	4ea215b1fcfd8271254460ed7c8f851b	130,97
.t21	847,872	4,096	1,024	0.99	4ea215b1fcfd8271254460ed7c8f851b	130,97
.t2	851,968	4,096	1,024	0.99	4ea215b1fcfd8271254460ed7c8f851b	130,97
.rdat	856,064	4,096	1,024	0.16	01cb5594d3b2c332c81196b782ade080	249,12
.rsrc	860,160	32,768	29,696	3.49	516c129e8650fabf0816b135bc84f78c	2,146,2
.SCY	892,928	8,192	5,120	5.57	6ed58308550db87c6d93949b69824f57	65,830

Imported APIs

DLL	Imported Functions
advapi32.dll	AddAccessAllowedAce, AdjustTokenPrivileges, AllocateAndInitializeSid, CheckTokenMembership, CloseServiceHandle, CreateServiceW, DeleteService, FreeSid, GetUserNameA, InitializeAcl

DLL	Imported Functions
dnsapi.dll	DnsFlushResolverCache
gdi32.dll	BitBlt, CreateCompatibleBitmap, CreateCompatibleDC, DeleteDC, GetObjectA, GetObjectW, SelectObject
kernel32.dll	CloseHandle, CopyFileA, CopyFileW, CreateDirectoryA, CreateEventW, CreateFileA, CreateFileW, CreateProcessW, CreateRemoteThread, CreateThread
oleaut32.dll	OleLoadPicture, SysAllocString, SysFreeString, VariantClear, VariantInit
psapi.dll	EnumProcesses, GetProcessImageFileNameW
shell32.dll	CommandLineToArgvW, SHChangeNotify, ShellExecuteW
user32.dll	BeginPaint, CreateDialogParamW, DialogBoxParamW, DispatchMessageW, EndDialog, EndPaint, FindWindowA, GetDC, GetMessageW, MessageBoxA
wininet.dll	HttpAddRequestHeadersA, HttpOpenRequestA, HttpSendRequestW, InternetCloseHandle, InternetConnectA, InternetOpenA, InternetQueryDataAvailable, InternetReadFile
ws2_32.dll	closesocket, connect, gethostbyname, htons, inet_addr, recv, send, shutdown, socket, WSACleanup
iphlpapi.dll	GetAdaptersInfo
msvcrt.dll	??2@YAPAXI@Z (operator new), ??3@YAXPAX@Z (operator delete), _close, _errno, _except_handler3, _lseek, _open, _read, _snwprintf, _strcmpi
ntdll.dll	NtClose, NtConnectPort, NtCreateSection, NtDelayExecution, NtQuerySystemTime, NtRequestWaitReplyPort, RtlNtStatusToDosError
ole32.dll	CoInitialize, CoInitializeEx, CoInitializeSecurity, CoUninitialize, CreateStreamOnHGlobal
combase.dll	CoCreateInstance

Analyzing Dump exe GHIDRA

```
/* WARNING: Function: __chkstk replaced with injection: alloca_probe */

void entry(void)

{
  WCHAR WVar1;
  char cVar2;
  undefined1 uVar3;
  ushort uVar4;
  HMODULE pHVar5;
  BOOL BVar6;
  uint uVar7;
  wchar_t *pwVar8;
  HINSTANCE pHVar9;
  LPWSTR pWVar10;
  LPWSTR *ppWVar11;
  undefined4 uVar12;
  HANDLE hObject;
  uint uVar13;
  uint uVar14;
  int iVar15;
  WCHAR *pWVar16;
  int *piVar17;
  short *psVar18;
  bool bVar20;
```

```
char *_Format;
DWORD DVar21;
undefined *puVar22;
char local_1044;
undefined1 local_1043 [2047];
undefined2 local_844;
undefined1 local_842 [518];
WCHAR local_63c;
undefined1 local_63a [518];
WCHAR local_434;
undefined1 local_432 [518];
char local_22c [264];
char local_124 [264];
undefined4 local_1c;
undefined4 local_18;
DWORD local_14;
DWORD local_10;
int local_c;
undefined1 auStack_8 [2];
char local_6;
char local_5;
short *psVar19;

local_14 = 0;
_auStack_8 = 0x1402044;
local_22c[0] = '\0';
memset(local_22c + 1,0,0x103);
local_124[0] = '\0';
memset(local_124 + 1,0,0x103);
local_1044 = '\0';
memset(local_1043,0,0x7ff);
local_434 = L'\0';
auStack_8 = (undefined1  [2])CONCAT11(0,auStack_8[0]);
local_c = 0;
local_18 = 0;
local_1c = 0;
memset(local_432,0,0x206);
local_844 = 0;
memset(local_842,0,0x206);
local_63c = L'\0';
memset(local_63a,0,0x206);
pHVar5 = GetModuleHandleA((LPCSTR)0x0);
BVar6 = VirtualProtect(pHVar5,0x400,0x40,&local_10);
if ((BVar6 != 0) && ((short)DAT_00414570 == 0x5a4d)) {
  piVar17 = &DAT_00414570;
  for (iVar15 = 0x100; iVar15 != 0; iVar15 = iVar15 + -1) {
    pHVar5->unused = *piVar17;
    piVar17 = piVar17 + 1;
    pHVar5 = pHVar5 + 1;
  }
  local_10 = 0;
  psVar18 = (short *)0x402030;
  do {
    psVar19 = psVar18 + -8;
    if (((*psVar19 == 0x5a4d) && (uVar7 = *(uint *)(psVar18 + 0x16), uVar7 < 0x1000)) &&
       (*(int *)(uVar7 + (int)psVar19) == 0x4550)) {
      pHVar5 = GetModuleHandleA((LPCSTR)0x0);
      local_10 = 0;
```

```
        if (*(short *)((int)&pHVar5[1].unused + uVar7 + 2) != 0) {
          piVar17 = (int *)((int)&pHVar5[0x41].unused + uVar7);
          do {
            *piVar17 = (int)psVar19 + (*piVar17 - (int)pHVar5);
            piVar17 = piVar17 + 10;
            local_10 = local_10 + 1;
          } while (local_10 < *(ushort *)((int)&pHVar5[1].unused + uVar7 + 2));
        }
        break;
      }
      local_10 = local_10 + 1;
      psVar18 = psVar19;
    } while (local_10 < 0x10000);
  }
  FUN_00401fad(u_SeDebugPrivilege_004139c4);
  FUN_00401fad(u_SeCreateGlobalPrivilege_004139e8);
  FUN_00406e34();
  if ((DAT_00425028 == 9) && (DAT_0042502c == 0)) {
    uVar7 = FUN_00409cd1();
    DAT_00425028 = uVar7 & 0xffff;
    DAT_0042502c = 0;
    if (DAT_00425028 == 0) {
      DAT_00425028 = FUN_0040a02d();
      DAT_0042502c = 0;
    }
  }
  GetModuleFileNameW((HMODULE)0x0,(LPWSTR)&DAT_004149b0,0x104);
  pwVar8 = wcsstr((wchar_t *)&DAT_004149b0,u_.bat_00413a18);
  if (pwVar8 == (wchar_t *)0x0) {
LAB_00402291:
    cVar2 = FUN_00401b98(&local_c);
    if (cVar2 == '\0') {
      uVar4 = FUN_00409f9c();
      DAT_0042d132 = uVar4 & 0xff;
      uVar3 = FUN_00403810();
      _auStack_8 = CONCAT13(uVar3,_auStack_8);
      CreateEventW((LPSECURITY_ATTRIBUTES)0x0,1,0,u_{9D723E3C-5DD2-43a4-A593-6C4327D_00413a70);
      local_14 = GetLastError();
      if ((local_14 == 0xb7) && (local_5 != '\0')) {
LAB_004022ea:
        DVar21 = 0;
        goto LAB_004022eb;
      }
      _auStack_8 = (uint3)(ushort)auStack_8;
      local_c = 0;
      pWVar10 = GetCommandLineW();
      ppWVar11 = CommandLineToArgvW(pWVar10,&local_c);
      if ((ppWVar11 != (LPWSTR *)0x0) && (1 < local_c)) {
        pWVar10 = ppWVar11[1];
        pwVar8 = u_shortcut_00413cb8;
        do {
          WVar1 = *pWVar10;
          bVar20 = (ushort)WVar1 < (ushort)*pwVar8;
          if (WVar1 != *pwVar8) {
LAB_00402344:
            iVar15 = (1 - (uint)bVar20) - (uint)(bVar20 != 0);
            goto LAB_00402349;
          }
```

```
        if (WVar1 == L'\0') break;
        WVar1 = pWVar10[1];
        bVar20 = (ushort)WVar1 < (ushort)pwVar8[1];
        if (WVar1 != pwVar8[1]) goto LAB_00402344;
        pWVar10 = pWVar10 + 2;
        pwVar8 = pwVar8 + 2;
      } while (WVar1 != L'\0');
      iVar15 = 0;
LAB_00402349:
      if (iVar15 == 0) {
        _auStack_8 = CONCAT12(1,auStack_8);
      }
    }
    if ((local_5 != '\0') && (DAT_00425134 != 0)) {
      local_c = 0;
      pWVar10 = GetCommandLineW();
      ppWVar11 = CommandLineToArgvW(pWVar10,&local_c);
      if ((ppWVar11 != (LPWSTR *)0x0) && (1 < local_c)) {
        pWVar10 = ppWVar11[1];
        pWVar16 = &DAT_00414004;
        do {
          WVar1 = *pWVar10;
          bVar20 = (ushort)WVar1 < (ushort)*pWVar16;
          if (WVar1 != *pWVar16) {
LAB_004023a2:
            iVar15 = (1 - (uint)bVar20) - (uint)(bVar20 != 0);
            goto LAB_004023a7;
          }
          if (WVar1 == L'\0') break;
          WVar1 = pWVar10[1];
          bVar20 = (ushort)WVar1 < (ushort)pWVar16[1];
          if (WVar1 != pWVar16[1]) goto LAB_004023a2;
          pWVar10 = pWVar10 + 2;
          pWVar16 = pWVar16 + 2;
        } while (WVar1 != L'\0');
        iVar15 = 0;
LAB_004023a7:
        if (iVar15 == 0) goto LAB_004023b8;
      }
      iVar15 = FUN_004038e3();
      if (iVar15 != 0) goto LAB_00402728;
    }
LAB_004023b8:
    iVar15 = 0;
    do {
      cVar2 = (&DAT_00425030)[iVar15];
      local_22c[iVar15] = cVar2;
      iVar15 = iVar15 + 1;
    } while (cVar2 != '\0');
    iVar15 = 0;
    do {
      cVar2 = (&DAT_00425138)[iVar15];
      local_124[iVar15] = cVar2;
      iVar15 = iVar15 + 1;
    } while (cVar2 != '\0');
    FUN_00406a7c(&DAT_00425250,&DAT_00425368);
    FUN_00407144(DAT_0042d132 != 0,0x1f,0,DAT_0042d134);
    FUN_00406a7c(&DAT_00425470,&DAT_00425368);
```

```
local_18 = FUN_004011dd(&local_18);
uVar12 = FUN_0040109f(&local_1c);
FUN_004075a0(uVar12,local_18);
FUN_00406a7c(&DAT_00425250,&DAT_00425368);
iVar15 = FUN_0040a839();
if (iVar15 == 2) {
  FUN_00407144(1,0x2a,0,2);
  FUN_004038e3();
}
else {
  if (0 < iVar15) {
    FUN_00407144(1,0x29,0,iVar15);
    do {
          /* WARNING: Do nothing block with infinite loop */
    } while( true );
  }
  FUN_00407144(0,0x2b,0,0);
  if (local_5 != '\0') {
    FUN_00407144(0,0x14,0,0);
  }
  FUN_00406a7c(local_22c,local_124);
  if (local_5 != '\0') {
    CreateEventW((LPSECURITY_ATTRIBUTES)0x0,1,0,u_{8B33EA89-2510-4223-8F24-02E6585_00413ac0);
    local_14 = GetLastError();
    if (local_14 == 0xb7) goto LAB_004022ea;
  }
  if ((DAT_00425574 != 0) && (cVar2 = FUN_00403810(), cVar2 != '\0')) {
    DAT_004149ac = CreateThread((LPSECURITY_ATTRIBUTES)0x0,0,FUN_00402d08,(LPVOID)0x0,0,
                          (LPDWORD)0x0);
  }
  if (local_5 == '\0') {
    bVar20 = local_6 == '\0';
    DVar21 = local_14;
    if (!bVar20) {
      FUN_00405c5f(1);
      DVar21 = 0;
    }
    FUN_00407144(bVar20,0x1e,0,DVar21);
  }
  iVar15 = FUN_00402cae();
  if (((iVar15 == 0) && (local_5 != '\0')) && (iVar15 = FUN_00406ab1(), iVar15 == 0xc)) {
    hObject = CreateEventW((LPSECURITY_ATTRIBUTES)0x0,1,0,
                        u_{9D723E3C-5DD2-43a4-A593-6C4327D_00413b10);
    iVar15 = FUN_0040ad81();
    if (iVar15 != 0) {
      FUN_00407144(0,0xe,0,0);
      Sleep(20000);
      CloseHandle(hObject);
      goto LAB_004022ea;
    }
    FUN_00407144(1,0xe,0,0);
    CloseHandle(hObject);
  }
  iVar15 = FUN_00402cae();
  if (iVar15 == 0) {
    if ((local_5 == '\0') && (iVar15 = FUN_00406ab1(), iVar15 == 0xc)) {
      FUN_00407144(1,0xe,0,0);
    }
```

```
        cVar2 = FUN_004035ee();
        _auStack_8 = CONCAT12(cVar2,auStack_8);
        if (local_5 != '\0') {
          FUN_00407144(cVar2 == '\0',0xc,0,0);
          if (local_6 != '\0') {
            FUN_00403874();
            iVar15 = FUN_00406ab1();
            if (iVar15 == 0xd) {
              if (DAT_0042567c != 0) {
                Sleep(DAT_0042567c * 1000);
              }
              FUN_004033b8(&DAT_00413a54);
            }
            cVar2 = FUN_004035ee();
            if (cVar2 == '\0') {
              FUN_00407144(0,0xd,0,0);
            }
            else {
              FUN_00407144(1,0xd,0,1);
              DialogBoxParamW((HINSTANCE)0x0,(LPCWSTR)&DAT_00000073,(HWND)0x0,FUN_00402d78,0);
              auStack_8 = (undefined1  [2])CONCAT11(1,auStack_8[0]);
            }
            local_c = 0;
            while( true ) {
              local_14 = FUN_00403701();
              if (local_14 == 0) break;
              local_c = local_c + 1;
              if (9 < local_c) {
                if (auStack_8[1] != '\0') {
                  FUN_00407144(1,0x24,0,1);
                }
                cVar2 = FUN_00406a43();
                if (cVar2 == '\0') {
                  FUN_0040304e();
                }
                else {
                  FUN_00403203();
                }
                FUN_00405c5f(0);
                puVar22 = &DAT_004149a6;
                _Format = s_http://www.bing.com/search?q={se_00413b70;
                goto LAB_00402707;
              }
              Sleep(60000);
              DialogBoxParamW((HINSTANCE)0x0,(LPCWSTR)&DAT_00000073,(HWND)0x0,FUN_00402d78,0);
            }
            if (auStack_8[1] != '\0') {
              FUN_00407144(0,0x24,0,0);
            }
            Sleep(30000);
            goto LAB_00402728;
          }
        }
      }
      cVar2 = FUN_00406a43();
      if (cVar2 == '\0') {
        FUN_00407144(1,0x11,0,0);
        uVar7 = FUN_00402e36();
```

```
        uVar13 = FUN_00402766();
        uVar14 = FUN_0040290c();
        if ((uVar7 & uVar13 & uVar14) != 0) {
          FUN_0040304e();
          puVar22 = &DAT_00414bb8;
          _Format = s_http://www.bing.com/search?q={se_00413bd0;
          goto LAB_00402707;
        }
      }
      else {
        FUN_00407144(0,0x11,0,0);
        uVar7 = FUN_00402e36();
        uVar13 = FUN_0040290c();
        if ((uVar13 & uVar7) != 0) {
          FUN_00403203();
          puVar22 = &DAT_004149a7;
          _Format = s_http://www.bing.com/search?q={se_00413ba0;
LAB_00402707:
          sprintf(&local_1044,_Format,puVar22);
          FUN_00404818();
          FUN_0040420b(&local_1044);
        }
      }
    }
  }
  else {
    FUN_00407144(1,0x14,0,local_c);
  }
 }
 else {
  FUN_004067f6(&local_844,9);
  wsprintfW(&local_63c,u_%s\%s.exe_00413a34,u_%TEMP%_00413a24,&local_844);
  ExpandEnvironmentStringsW(&local_63c,&local_434,0x104);
  CopyFileW((LPCWSTR)&DAT_004149b0,&local_434,0);
  pHVar9 = ShellExecuteW((HWND)0x0,u_open_00413a48,&local_434,(LPCWSTR)0x0,(LPCWSTR)0x0,3);
  if ((int)pHVar9 < 0x21) {
    local_14 = GetLastError();
    goto LAB_00402291;
  }
 }
LAB_00402728:
 FUN_00406992(&DAT_004149b0);
 if ((DAT_00425022 != '\0') && (iVar15 = WSACleanup(), iVar15 == 0)) {
  DAT_00425022 = '\0';
 }
 if (local_5 == '\0') {
  FUN_004033b8(&DAT_00413c00);
 }
 FUN_00403874();
 DVar21 = local_14;
LAB_004022eb:
                    /* WARNING: Subroutine does not return */
 ExitProcess(DVar21);
}
```

High-Level Execution Flow

Stage	Description
Initialization	Clears stack buffers, initializes local variables, prepares wide/ANSI buffers
Self-modification	Changes memory protection and patches its own PE image in memory
Privilege Setup	Enables high-risk privileges (e.g., SeDebugPrivilege)
Environment Checks	Single-instance enforcement, command-line parsing, execution context checks
Persistence / Relaunch	Copies itself to `%TEMP%` and relaunches if required
Core Logic	Executes main malicious routines and decision logic
Networking / C2	Builds URLs and triggers HTTP-based communication
Cleanup & Exit	Cleans up Winsock, threads, and exits process

Self-Modification & Anti-Analysis

Behavior	Evidence in Code	Purpose
Memory protection change	`VirtualProtect(pHVar5, 0x400, 0x40, &local_10)`	Allows code modification
PE header validation	Checks `MZ` (0x5A4D) and `PE` (0x4550)	Confirms valid executable
Runtime relocation fixups	Manual adjustment of pointers	Anti-dump / unpacking behavior
Stack probing injection	`__chkstk replaced with injection: alloca_probe`	Obfuscation / compiler evasion

Privilege Escalation & Security Bypass

Action	Function / Indicator	Intent
Enable debug privilege	`FUN_00401fad("SeDebugPrivilege")`	Access protected processes
Enable global object creation	`FUN_00401fad("SeCreateGlobalPrivilege")`	System-wide IPC / events
NT-level API usage	Low-level PE and memory access	Bypass user-mode hooks

Single-Instance & Mutex-Like Logic

Technique	Details
Named event objects	`CreateEventW("{GUID}")`
Instance detection	Checks `GetLastError() == ERROR_ALREADY_EXISTS (0xB7)`
Behavior on duplicate	Early exit or altered execution path

Command-Line & Execution Mode Handling

Check	Behavior
Command-line parsing	`GetCommandLineW`, `CommandLineToArgvW`
Special arguments	Compares against hardcoded Unicode strings
Execution flags	Alters control flow via `local_5`, `local_6`, `_auStack_8`

Persistence & Self-Replication

Technique	Evidence
Copy to TEMP	`ExpandEnvironmentStringsW("%TEMP%")`, `CopyFileW`
Rename executable	`%TEMP%\{random}.exe`
Relaunch	`ShellExecuteW("open", new_path)`
Fallback execution	Retries original logic if relaunch fails

Threading & Background Execution

Behavior	Function
Background worker thread	`CreateThread(FUN_00402d08)`
Conditional execution	Triggered only if specific flags are set
Long-running loops	Infinite / delayed execution with `Sleep()`

Network & C2-Like Activity

Indicator	Details
Hardcoded URLs	`http://www.bing.com/search?q={...}`
Dynamic URL building	`sprintf(local_1044, format, data)`
HTTP trigger functions	`FUN_0040420b` , `FUN_00404818`
Purpose	Likely C2 beaconing or covert data exfiltration

User Interaction & Social Engineering

Feature	Evidence
Dialog popups	`DialogBoxParamW`
Message-based logic	Repeated dialogs with delays
User influence	Possibly used for decoy UI or interaction gating

Anti-Debugging & Delay Tactics

Technique	Evidence
Long sleep loops	`Sleep(60000)` , `Sleep(30000)` , `Sleep(20000)`
Retry counters	Loops capped at 9–10 iterations
Conditional exits	Based on runtime environment

Cleanup & Termination

Action	Function
Network cleanup	`WSACleanup()`
Resource cleanup	`CloseHandle()`
Final exit	`ExitProcess(local_14)`

Overall Malware Capability Assessment

Capability	Present
Self-modifying code	✅
Privilege escalation	✅
Persistence	✅
Anti-analysis	✅
Network communication	✅
User interaction	✅
Multi-threading	✅

The analyzed `entry()` function demonstrates advanced malware loader behavior, including runtime PE patching, privilege escalation, persistence through self-replication, single-instance enforcement, and network-based command execution. The use of legitimate services (bing.com) suggests C2 camouflage, while extensive delay loops and NT-level operations indicate evasion-aware design.
